Rolling Cognitive State Maintenance: Why Structured Carry-Forward May Matter More Than Raw Context Length

Modern language models increasingly support enormous context windows. A model can now receive tens of thousands, and in some cases hundreds of thousands, of tokens in a single conversation. This has encouraged a simple assumption: if the model can retain more of the conversation, then it should become a better long-running agent.

That assumption is incomplete.

A large context window can preserve history, but preserving history is not the same as preserving a coherent working state. In a long-running agentic task, the conversation gradually accumulates far more than useful information. It accumulates abandoned plans, corrected assumptions, obsolete file paths, failed experiments, speculative branches, duplicate explanations, tool outputs, revised requirements, old configurations, new configurations, intermediate reasoning, provisional conclusions, and conclusions that were later overturned.

All of this may remain technically available to the model.

But not all of it should remain equally influential.

The deeper problem of long-context agency is therefore not simply memory. It is control over memory.

The central question becomes:

> Which information from the entire history should actually govern the model's next action?

This is the problem that the Jack architecture is designed to address.

Jack approaches long-context operation through a process that can be described as rolling cognitive state maintenance. Instead of assuming that an ever-growing unstructured transcript will continue to function as reliable working memory, Jack repeatedly reconstructs the most important operative

information and carries it forward in structured XML representations. Those representations are placed close to the model's next output, allowing the current state of the task to remain prominent even as the historical transcript grows increasingly large and complicated.

The result is a fundamentally different approach to long-context reasoning.

It replaces passive retention with active state maintenance.

Long Context Is Not the Same as Working Memory

Consider a software-engineering agent operating for many hours.

Early in the session, it may establish:

```
project path = /project/v1
```

Later, that path is discovered to be wrong:

```
project path = /project/v2
```

The original statement has not disappeared from the context. The corrected statement simply appears later.

The same thing can happen with models, configurations, test results, artifact identities, plans, and conclusions.

An experiment may initially suggest:

```
retrieval enabled = failure
```

A later controlled experiment may show that retrieval was irrelevant and that quantization was the actual variable.

Both conclusions can remain in context.

A plan may initially require one architecture, then later be rejected and replaced.

Both plans remain visible.

A model may therefore have perfect textual recall and still behave incorrectly.

It can remember the obsolete path and the correct path and still use the obsolete one.

It can remember that one configuration was tested and another was not tested and still transfer the result between them.

It can remember that a hypothesis was rejected while continuing to reason as though it were true.

This demonstrates an important distinction between historical recall and operative recall.

> Historical recall asks:
>
> Can the model retrieve what happened?

> Operative recall asks:
>
> Can the model determine what currently matters and continue acting according to it?

For a serious agent, the second question is much more important.

The Problem of Unstructured Context

Long conversations are inherently noisy.

As an agent works, its transcript begins to contain multiple layers of cognitive residue:

- hypotheses that were investigated and rejected,
- hypotheses that remain possible,
- intermediate calculations,
- obsolete requirements,
- updated requirements,
- earlier plans,
- revised plans,
- unsuccessful code,

- corrected code,
- invalid test results,
- verified test results,
- commentary,
- tool output,
- uncertainty,
- unrelated side discussions,
- and repeated restatements of the same information.

An ordinary transformer can still attend to all of this information, but availability alone does not guarantee that the right information will dominate the next generation.

The model must continually distinguish between historical information and controlling information.

This distinction becomes increasingly difficult as the trajectory grows.

The problem is therefore not simply that an old fact is "far away." Transformers can often retrieve distant information successfully. The more serious issue is that the model must choose among a large number of competing representations, many of which remain linguistically plausible.

A stale conclusion can look almost identical to the current conclusion.

An obsolete path may differ by only a few characters.

A failed configuration may differ from a successful configuration by one parameter.

A rejected plan may still be logically coherent.

The model is not searching an empty memory. It is searching a memory full of plausible competitors.

That creates a signal-to-noise problem.

Jack's Alternative: Carry the Operative State Forward

Jack's solution is to continually reconstruct the information that should remain behaviorally active.

At the end of a turn or reasoning cycle, the model does not simply produce an answer and leave all important state scattered across the preceding transcript.

It places critical information into structured Jack XML.

The next stage therefore begins with both the historical context and a compact representation of what the architecture has determined should remain operative.

The process can continue repeatedly:

```
TURN 1
full context
    ↓
reasoning
    ↓
Jack identifies operative state
    ↓
structured XML near output
TURN 2
previous history
+ previous Jack state
+ new information
    ↓
reasoning / execution
    ↓
state revised
    ↓
new Jack XML near output
TURN 3
larger history
+ refreshed operative state
    ↓
new evidence
    ↓
superseded facts separated
    ↓
new Jack XML
...
TURN N
very large historical context
+
recent structured representation
of what still controls the task
```

This creates a form of rolling working state.

The complete conversation is still available when historical evidence is needed. Jack does not need to destroy the past.

But the model is no longer forced to treat the full transcript as its only representation of the present.

Frontier Rebinding

One of the most important aspects of this architecture is where the structured state appears.

Jack places the XML immediately before the final output or next action.

This creates what can be called frontier rebinding.

Imagine that native reasoning produces 6,000 tokens.

The model may correctly establish an important fact halfway through the reasoning:

> Configuration D has never been tested.

But another 3,000 tokens may follow.

Those tokens may include alternative hypotheses, related configurations, edge cases, calculations, and discarded ideas.

By the time the final answer is generated, the decisive fact is no longer near the generation frontier.

Jack reconstructs it:

```
<anchor_fact ID="CONFIG_D_STATUS">
UNTESTED
</anchor_fact>
```

Now the fact that must govern the answer appears immediately before the answer.

The architecture is therefore doing more than summarization.

It is taking important cognitive variables from deep history and repositioning them where they are maximally relevant to the next generation.

This matters for both long conversations and long native reasoning traces.

The concept can be summarized as:

> Do not merely leave important information somewhere in memory. Rebind it near action.

Structure Matters as Much as Recency

Simply repeating the last several facts would not provide the same benefit.

Jack uses structured tags because different pieces of information have different cognitive roles.

For example:

- `<workspace_state>` can describe the currently active task state.
- `<grounded_source>` can distinguish evidence from assumption.
- `<anchor_fact>` can isolate exact values, identities, paths, configuration states, or conclusions.
- `<pitfall_check>` can preserve active failure boundaries and counterarguments.
- `<deterministic_check>` can distinguish verified external results from probabilistic reasoning.

This structure prevents the carry-forward state from becoming another undifferentiated paragraph.

A model reading:

```
We tested Q4 and Q5 and retrieval was on for one test and off for another and maybe retrieval
caused the error although later Q5 passed and configuration D hasn't been tested.
```

must reconstruct the relationships.

A structured representation can instead state:

```
<anchor_fact ID="CONFIG_A">
Q4 / retrieval ON / FAILED
</anchor_fact>

<anchor_fact ID="CONFIG_B">
Q4 / retrieval OFF / FAILED
</anchor_fact>

<anchor_fact ID="CONFIG_C">
Q5 / retrieval OFF / PASSED
</anchor_fact>
```

```
<anchor_fact ID="CONFIG_D">
Q5 / retrieval ON / UNTESTED
</anchor_fact>
```

The same information is now easier to distinguish.

The architecture is therefore using both semantic organization and contextual proximity.

Why Ordinary Summarization Is Not Enough

One might argue that this is simply conversation summarization.

It is not.

Ordinary summaries are often aggressively lossy.

A generic summary system tends to preserve the narrative center of a conversation while discarding what appears secondary.

For a human reader, that may be desirable.

For an autonomous agent, supposedly secondary details may be exactly what prevents failure.

A summary might transform:

```
Configuration D has not been tested. Do not infer its behavior from configuration C.
```

into:

```
Q5 appears to resolve the issue.
```

That sounds reasonable and is catastrophically different.

Similarly, a summary may discard:

- exact filenames,
- precise version numbers,
- rejected branches,
- provenance,

- unresolved uncertainty,
- counterexamples,
- configuration boundaries,
- negative evidence,
- deterministic verification status.

Jack instead aims at lossless semantic compression of operative information.

Compression is still necessary. An agent cannot continually duplicate its entire context.

But the compression target is different.

The objective is not:

```
Produce a shorter version of the conversation.
```

The objective is:

> ```
> Preserve the variables necessary to continue acting correctly.
> ```

That is a much more demanding standard.

Carry-Forward as State Transition

The most useful way to understand the architecture is as a sequence of state transitions.

Let the full historical context at turn t be H_t.

Let Jack's operative state be S_t.

New observations arrive as O_{t+1}.

Rather than asking the model to derive everything directly from the full history every time, the agent effectively performs:

$$S_{t+1} = F(S_t, O_{t+1}, H_t)$$

where F reconstructs the next authoritative state.

The historical transcript remains available, but the operative state is repeatedly renewed.

This is conceptually similar to how many successful computational systems operate.

- Databases preserve transaction history while maintaining a current state.
- Operating systems preserve logs while maintaining current process state.
- Version-control systems preserve old commits while identifying the active revision.
- Humans preserve autobiographical memory while maintaining a much smaller working memory.

An effective AI agent needs something analogous.

Treating the complete conversation as both archive and working memory places two very different responsibilities on the same unstructured representation.

Jack begins to separate them.

Adaptive Reasoning Makes This More Powerful

This becomes particularly important when Jack is used inside an agentic workflow that selectively enables and disables native thinking.

Native thinking can be enabled when the system encounters:

- a difficult architectural decision,
- a novel bug,
- conflicting evidence,
- uncertainty,
- a high-risk action,
- or an independent review stage.

During those stages, the model can search broadly.

It can generate hypotheses, explore alternatives, construct counterexamples, and perform exhaustive falsification.

Once an operative state has been established, native thinking can be disabled for routine execution.

But Jack does not disappear.

The XML structures remain active as attention anchors.

The model still receives:

- the authoritative objective,
- active constraints,
- current artifact identity,
- relevant evidence,
- known pitfalls,
- and the state that should govern execution.

This distinction is critical.

> Thinking OFF is not the absence of cognitive architecture. It is reduced native deliberation under maintained Jack alignment.

The agent can therefore spend expensive reasoning compute when it creates value without forcing every routine action to regenerate the entire reasoning process.

Why This Could Compound Over Many Turns

The greatest benefit may not appear in a single prompt.

It may emerge after 20, 50, 100, or more agentic transitions.

Without structured carry-forward, noise accumulates.

Every turn adds more historical material.

If the model must repeatedly infer current state from that entire history, the probability of drift can increase.

With rolling state maintenance, every turn is also an opportunity to cleanly re-establish the operative state.

Instead of allowing important information to drift progressively deeper into the transcript, Jack continually moves it forward.

Instead of allowing obsolete state to remain undifferentiated from active state, Jack can explicitly classify it.

Instead of allowing uncertainty to silently become certainty through repetition, Jack can carry the uncertainty forward as uncertainty.

Instead of asking each future inference to rediscover the authority hierarchy from scratch, the hierarchy can remain explicitly represented.

This creates the possibility of compounding stability rather than compounding noise.

That may be one of the most important implications of the architecture.

A Different View of Agent Memory

The broader lesson is that AI memory should not be understood only as storage.

> ```
> An agent needs at least two kinds of memory:
> historical memory, which preserves what happened,
> and
> operative memory, which preserves what currently controls behavior.
> ```

Long context provides increasingly powerful historical memory.

Jack is an attempt to engineer the second.

The architecture therefore asks a question that becomes increasingly important as model context windows grow:

> What good is remembering everything if the model cannot reliably determine what matters now?

The answer is not necessarily more tokens.

It may be better organization of the tokens already available.

Conclusion

The central hypothesis behind Jack's long-context architecture is straightforward:

> Long-running agent performance should improve when important operative information is repeatedly reconstructed, structurally anchored, and carried forward near the generation frontier instead of remaining dispersed inside an increasingly noisy unstructured transcript.

This transforms the context from a passive record into an actively maintained cognitive environment.

Jack does not discard history.

It does not assume that old information is useless.

Instead, it separates the existence of historical information from the authority of current information.

Through repeated XML carry-forward, attention anchoring, state reconstruction, configuration separation, grounding, falsification, and frontier rebinding, the architecture attempts to ensure that the information most relevant to the next decision remains both available and behaviorally salient.

That is why the significance of Jack is not merely that XML tags appear before an answer.

The deeper idea is rolling cognitive state maintenance.

Each turn becomes an opportunity to reconstruct the present from the past.

Each transition refreshes the state that should control the future.

The transcript can continue growing, but the agent does not have to become progressively buried beneath its own history.

And that may be one of the central requirements for reliable long-horizon AI agents:

Preserve the full history, but continually carry the right state forward.